



hallo werde aus meiner suche bisher nicht richtig schlau, was ist denn jetzt der unterschied zwischen url und formular? also request args und request form. wie funktioniert das im backend ? was sind die mechanismen ?

Der Kernunterschied ist:

- `request.args` liest Daten aus der URL (Query-Parameter).^[1] ^[2]
- `request.form` liest Daten aus dem Request-Body, die über ein HTML-Formular per POST gesendet wurden.^[2] ^[1]

Ich erkläre das Schritt für Schritt, z.B. für Flask, aber die Mechanismen sind allgemein HTTP-Standard.

1. Was ist „URL“ vs. „Formular“?

- URL-Parameter (Query-String): Alles was hinter einem ? in der URL steht, z.B. `/search?q=python&page=2`.^[1] ^[2]
- Formular-Daten: Wenn du ein HTML-Formular hast (`<form method="post">`), werden die Eingaben beim Abschieken normalerweise im Body der HTTP-Anfrage gesendet, nicht in der URL.^[3] ^[1]

Beispiel:

```
GET /search?q=python&page=2 HTTP/1.1
Host: example.com
```

Hier stecken `q` und `page` in der URL → `request.args`.^[2] ^[1]

```
POST /login HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

username=alice&password=secret
```

Hier stecken `username` und `password` im Body → `request.form`.^[4] ^[1] ^[2]

2. request.args im Backend

- `request.args` enthält die Query-Parameter aus der URL als eine Art Dictionary (MultiDict).
[\[4\]](#) [\[1\]](#) [\[2\]](#)
- Egal ob der Request GET oder POST ist: Wenn etwas in der URL hinter `?` steht, findest du es in `request.args`.
[\[5\]](#) [\[1\]](#) [\[2\]](#)

In Flask zum Beispiel:

```
@app.route("/search")
def search():
    term = request.args.get("q")          # aus /search?q=python
    page = request.args.get("page")       # aus /search?page=2
```

Mechanismus im Hintergrund:

- Der Webserver liest den HTTP-Request.
- Das Framework parst die URL und trennt Pfad (`/search`) und Query-String (`q=python&page=2`).
[\[1\]](#) [\[2\]](#)
- Es wandelt den Query-String in Schlüssel/Wert-Paare um und legt sie in `request.args`.
[\[2\]](#) [\[4\]](#)
[\[1\]](#)

3. request.form im Backend

- `request.form` enthält Daten, die im Request-Body stehen und als „Form-Data“ kodiert sind (typisch `application/x-www-form-urlencoded` oder `multipart/form-data`).
[\[6\]](#) [\[4\]](#) [\[1\]](#) [\[2\]](#)
- Klassischer Fall: HTML-Formular mit `method="POST"` wird abgeschickt.
[\[3\]](#) [\[1\]](#)

In Flask:

```
@app.route("/login", methods=["POST"])
def login():
    username = request.form.get("username")
    password = request.form.get("password")
```

Mechanismus im Hintergrund:

- Der Webserver liest den Body des HTTP-Requests.
- Wenn der Content-Type ein Formular-Encoding ist, parst das Framework den Body in Schlüssel/Wert-Paare.
[\[6\]](#) [\[4\]](#) [\[1\]](#) [\[2\]](#)
- Diese Paare landen in `request.form`.
[\[4\]](#) [\[1\]](#) [\[2\]](#)

Wenn du statt Formulardaten JSON schickst (Content-Type: `application/json`), sind die Daten **nicht** in `request.form`, sondern du nutzt `request.get_json()` bzw. `request.json`.
[\[7\]](#) [\[6\]](#) [\[1\]](#) [\[4\]](#)

4. GET vs. POST und Missverständnisse

Viele sagen:

- GET → `request.args`
- POST → `request.form`

Genauer ist:

- GET hat normalerweise **keinen** Body, deshalb gibt es nur URL-Parameter → `request.args`.^[1]
^[2]
- POST kann sowohl URL-Parameter als auch Body-Daten haben:
 - URL-Parameter → `request.args`
 - Formulardaten im Body → `request.form`^[8] ^[5] ^[2] ^[4] ^[1]

Also: Ein POST-Request zu `/user?id=123` mit einem Formular-Body hat `id` in `request.args` und die restlichen Felder im `request.form`.^[5] ^[8] ^[2] ^[4] ^[1]

5. Was passiert technisch im Backend?

Die Schritte sind bei den meisten Frameworks ähnlich:

1. HTTP-Request kommt an (Methode, URL, Header, Body).

2. Framework parst:

- Pfad (z.B. `/user/123`) → oft als Funktionsparameter (`def show_user(id):`).^[1]
- Query-String (nach `?`) → `request.args`.^[2] ^[4] ^[1]
- Body:
 - Wenn Content-Type Formular-Encoding → `request.form`.^[6] ^[4] ^[2] ^[1]
 - Wenn Content-Type `application/json` → `request.get_json()` / `request.json`.^[7] ^[6] ^[4]
^[1]
 - Wenn `multipart/form-data` mit Datei → Textfelder in `request.form`, Dateien in `request.files`.^[2] ^[1]

3. Das alles hängt dann an einem Request-Objekt, das du in deinem Handler verwendest.^[4] ^[1]
^[2]

6. Kurz zusammengefasst

- URL vs. Formular ist nicht „zwei völlig verschiedene Welten“, sondern nur zwei verschiedene Orte, an denen Daten im HTTP-Request stecken: Query-String vs. Body.^[1] ^[2]
- `request.args`: alles, was in der URL nach `?` steht.^[4] ^[2] ^[1]
- `request.form`: alles, was als klassisches HTML-Formular im Body gesendet wird.^[6] ^[2] ^[4] ^[1]
- JSON-Payloads und Dateien haben wieder eigene Zugriffswege (`request.get_json()`, `request.files`).^[7] ^[6] ^[2] ^[4] ^[1]

Wenn du magst, kannst du mir kurz sagen, mit welchem Framework du arbeitest (Flask, Django, FastAPI, Express, ...), dann kann ich dir den gleichen Ablauf mit einem konkreten Beispiel aus „deiner“ Welt zeigen.



1. <https://velog.io/@sangmin7648/오늘의-배움-045>
2. <https://www.bilibili.com/read/cv13532588/>
3. <https://web.dev/learn/forms/form?hl=de>
4. https://www.learnthatstack.com/interview-questions/backend_development/flask/what-s-the-difference-between-request-form-request-args-and-request-get-json-19389
5. https://blog.csdn.net/weixin_42894309/article/details/86532953
6. <https://stackoverflow.com/questions/23326368/sending-json-to-flask-request-args-vs-request-form?noredirect=1>
7. <https://www.iditect.com/faq/python/sending-json-to-flask-requestargs-vs-requestform.html>
8. <https://www.cnblogs.com/haoxinchen/p/9036366.html>
9. <https://form.taxi/de/knowledge/html-formular-erstellen>
10. <https://blog.csdn.net/wenpy/article/details/99075518>